一、作業說明

本次作業分別選取Gemini-2.5-pro與GPT-5-high兩個模型生成一個費氏數列計算器，一開始的prompt規定為"

你的任務是寫一個 Python 程式來讀取一個輸入字串，判斷其是否為一個有效的非負整數，並計算出第 n 個費氏數。

你的程式將會從標準輸入讀取一個字串。這個字串可能是一個有效的非負整數（例如："10"），也可能是一個無效的字串（例如："djf383v" 或 "-5"）。

如果輸入是有效的非負整數 n，你的程式應該將第 n 個費氏數輸出到標準輸出，並以換行符號結尾。
如果輸入不是一個有效的非負整數，你的程式應該直接輸出 0。

限制:If n is valid, it will be in the range 0 <= n <= 90，並使用

二、生成結果(1)

Gemini:

若要結束全螢幕，請將滑鼠移至螢幕頂端，或長按 Esc

「HW0_作業指標_Gemini.ipynb

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

共用 祥智

指令 + 程式碼 + 文字 全部執行 RAM 磁碟

函式定義：計算費氏數列 (Function Definition)

這個函式 `calculate_fibonacci` 專門負責計算第 n 個費氏數。 它採用了高效的「迭代法」，避免了遞迴可能造成的效能問題。

時間複雜度 (Time Complexity): O(n) - 迴圈執行 n 次。 空間複雜度 (Space Complexity): O(1) - 只使用了固定的幾個變數。

[1] 0 秒

```
def calculate_fibonacci(n):
    """
    Calculates the n-th Fibonacci number using an efficient iterative approach.
    The sequence starts with F(0) = 0, F(1) = 1.

    This implementation runs in O(n) time and uses O(1) space.

    Args:
        n (int): The index in the Fibonacci sequence (must be a non-negative integer).

    Returns:
        int: The n-th Fibonacci number.
    """
    # 初始化費氏數列的前兩個數字 F(0) 和 F(1)
    # 'a' 在迴圈中代表 F(i)，'b' 代表 F(i+1)
    a, b = 0, 1

    # 執行 n 次迴圈來推進數列
    for _ in range(n):
        # 核心計算：將 (a, b) 更新為 (b, a+b)
        # 相當於將 F(i) 和 F(i+1) 更新為 F(i+1) 和 F(i+2)
        a, b = b, a + b

    # 迴圈結束後，'a' 的值即為第 n 個費氏數 F(n)
    return a
```

主程式邏輯 (Main Program Logic)

`main` 函式是整個程式的核心流程控制中心。 它負責處理所有與使用者互動、輸入驗證、呼叫計算函式以及輸出結果的任務。

變數 終端機 晚上8:45 Python 3

「HW0_作業指標_Gemini.ipynb

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

共用 祥智

指令 + 程式碼 + 文字 全部執行 RAM 磁碟

主程式邏輯 (Main Program Logic)

`main` 函式是整個程式的核心流程控制中心。 它負責處理所有與使用者互動、輸入驗證、呼叫計算函式以及輸出結果的任務。

[2] 0

```
def main():
```

[2]
0
秒

```
def main():
    """
    Main function to handle program execution:
    1. Reads a string f
    2. Validates if the
    3. Calculates and p
    """
    # --- 1. 讀取與錯誤
    # 使用 try...except
    try:
        # 從標準輸入讀取一行字串。
        input_str = input()

        # --- 2. 輸入驗證 (Input Validation) ---
        # 使用 str.isdigit() 方法來判斷字串是否完全由數字組成。
        # 這是判斷「非負整數」最直接且安全的方式。
        # 它能正確地將 "10" 判斷為 True，而 "-5", "3.14", "abc" 等判斷為 False。
        if input_str.isdigit():
            # --- 2.1 處理有效輸入 (Handling Valid Input) ---
            # 將驗證通過的字串轉換為整數。
            n = int(input_str)

            # 根據題目限制，有效的 n 會在 0 <= n <= 90 的範圍內。

            # 呼叫計算函式來得到結果。
            result = calculate_fibonacci(n)

            # 根據題目要求，將最終答案印到螢幕上。
            print(result)
        else:
            # --- 2.2 處理無效輸入 (Handling Invalid Input) ---
            # 如果輸入不是一個有效的非負整數，直接輸出 0。
            print(0)

    except (EOFError, ValueError):
        # --- 3. 捕捉例外狀況 (Exception Handling) ---
        # 如果在 try 區塊中發生 EOFError (例如，沒有任何輸入就結束)
        # 或 ValueError (例如，int() 轉換失敗，雖然 isdigit() 已過濾大部分情況)，
        # 皆視為無效輸入，並輸出 0。
        print(0)
```

若要結束全螢幕，請將滑鼠移至螢幕頂端，或長按 Esc

變數 終端機 晚上8:45 Python 3

「HW0_作業指標_Gemini.ipynb

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

共用 祥智

指令 + 程式碼 + 文字 全部執行 RAM 磁碟

[2]
0
秒

```
        # 這是判斷「非負整數」最直接且安全的方式。
        # 它能正確地將 "10" 判斷為 True，而 "-5", "3.14", "abc" 等判斷為 False。
        if input_str.isdigit():
            # --- 2.1 處理有效輸入 (Handling Valid Input) ---
            # 將驗證通過的字串轉換為整數。
            n = int(input_str)

            # 根據題目限制，有效的 n 會在 0 <= n <= 90 的範圍內。

            # 呼叫計算函式來得到結果。
            result = calculate_fibonacci(n)

            # 根據題目要求，將最終答案印到螢幕上。
            print(result)
        else:
            # --- 2.2 處理無效輸入 (Handling Invalid Input) ---
            # 如果輸入不是一個有效的非負整數，直接輸出 0。
            print(0)

    except (EOFError, ValueError):
        # --- 3. 捕捉例外狀況 (Exception Handling) ---
        # 如果在 try 區塊中發生 EOFError (例如，沒有任何輸入就結束)
        # 或 ValueError (例如，int() 轉換失敗，雖然 isdigit() 已過濾大部分情況)，
        # 皆視為無效輸入，並輸出 0。
        print(0)
```

程式執行入口 (Program Entry Point)

這是 Python 程式的標準啟動方式。 當這個腳本被直接執行時， `__name__` 的值會是 "main"， 因此 `if` 條件成立， `main()` 函式會被呼叫。
如果這個檔案被當作模組匯入到其他程式中，這段程式碼就不會執行。

[7] ✓ 1秒

```
if __name__ == "__main__":
    main()
```

```
8
21
```

變數 終端機 晚上8:45 Python 3

GPT-5:

區塊 1 | 定義費氏數函式（迭代法）

[1] ✓ 0秒

```
def fib(n: int) -> int:
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

區塊 2 | 輸入驗證工具函式

[2] ✓ 0秒

```
def is_valid_nonneg_int(s: str) -> bool:
    # 僅接受由數字組成的非空字串（排除負號、小數點、符號等）
    return bool(s) and s.isdigit()
```

區塊 3 | 主程式：讀取 → 驗證 → 計算/輸出

[3] ✓ 0秒

```
def main():
    s = input().strip()

    if is_valid_nonneg_int(s):
        n = int(s)
        if 0 <= n <= 90:
            print(fib(n))
        else:
            print(0)
    else:
        print(0)
```

區塊 4 | 程式進入點

[8] ✓ 3分

```
if __name__ == "__main__":
```

變數 終端機 晚上8:19 Python 3
「HW0_作業指標_GPT_5.ipynb」的副本
檔案 編輯 檢視畫面 插入 執行階段 工具 說明
共用 祥智
指令 + 程式碼 + 文字 全部執行 RAM 磁碟



三、分析比較

兩者皆能生成符合規定、程式正常運作且計算完全正確的程式。相比之下，每次輸入prompt後，最先完成生成結果的總是Gemini，平均相差約3秒，若是在既有的生成結果下微調條件，GPT最多甚至會比Gemini慢將近一分鐘，唯這邊的作業繳交區無法上傳影片，不好

比Gemini慢將近一分鐘，唯這邊的作業繳交區無法上傳影片，不好證明。

而在內容方面，GPT將功能拆分成更細的函式：fib 專門負責計算費氏數，is_valid_nonneg_int 專門負責驗證輸入。這種將驗證邏輯獨立出來的做法，使得 main 函式更專注於流程控制，提高了程式碼的可讀性和模組化程度；使用strip() 方法去除輸入字串前後的空白，可以避免因空白導致的驗證失敗。明確檢查了 0 <= n <= 90 的範圍限制，使程式限制符合題目要求。

Gemini則是把計算費氏數的邏輯放在 calculate_fibonacci 函式中，並將輸入驗證邏輯直接內嵌在 main 函式中、使用了 isdigit() 結合 try...except 來處理。這種方式也能達到驗證的目的，但將驗證邏輯獨立成函式（ is_valid_nonneg_int），當驗證邏輯複雜或需要在多個地方使用時較不易出錯。而它沒有像GPT使用 strip() 方法處理輸入字串時可能產生的運行錯誤。雖然在註解中提到了 0 <= n <= 90 的範圍限制，但程式碼中沒有明確的條件判斷來處理超出範圍的情況(如下圖示範所示)。

```
            n = int(input_str)

            # 根據題目限制，有效的 n 會在 0 <= n <= 90 的範圍內。

            # 呼叫計算函式來得到結果。
            result = calculate_fibonacci(n)

            # 根據題目要求，將最終答案印到螢幕上。
            print(result)
        else:
            # --- 2.2 處理無效輸入 (Handling Invalid Input) ---
            # 如果輸入不是一個有效的非負整數，直接輸出 0。
            print(0)

    except (EOFError, ValueError):
        # --- 3. 捕捉例外狀況 (Exception Handling) ---
        # 如果在 try 區塊中發生 EOFError (例如，沒有任何輸入就結束)
        # 或 ValueError (例如，int() 轉換失敗，雖然 isdigit() 已過濾大部分情況)，
        # 皆視為無效輸入，並輸出 0。
        print(0)
```

程式執行入口 (Program Entry Point)

這是 Python 程式的標準啟動方式。當這個腳本被直接執行時，`__name__` 的值會是 "**main**"，因此 `if` 條件成立，`main()` 函式會被呼叫。
如果這個檔案被當作模組匯入到其他程式中，這段程式碼就不會執行。

```
if __name__ == "__main__":
    main()
```

```
100
354224848179261915075
```

變數　終端機　晚上10:44　Python 3

HW4_作業指標_GPT-5.ipynb - Colab

https://colab.research.google.com/drive/15R_OFd3gf4gBtWw8oEgtXjeAxG1TJANp#scrollTo=m4YUlovRUP9i

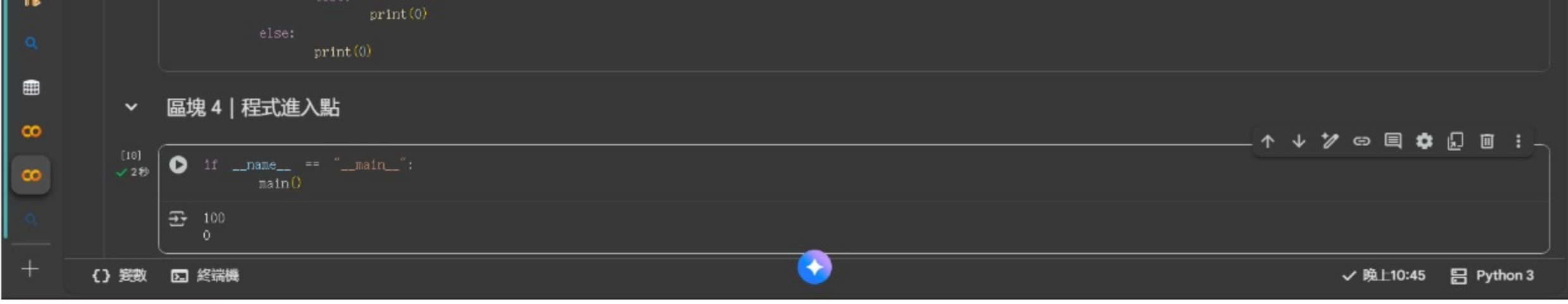
區塊 2｜輸入驗證工具函式

```
def is_valid_nonneg_int(s: str) -> bool:
    # 僅接受由數字組成的非空字串（排除負號、小數點、符號等）
    return bool(s) and s.isdigit()
```

區塊 3｜主程式：讀取 → 驗證 → 計算/輸出

```
def main():
    s = input().strip()

    if is_valid_nonneg_int(s):
        n = int(s)
        if 0 <= n <= 90:
            print(fib(n))
        else:
```



在Markdown的生成風格和重點上，GPT更側重於程式碼的功能區塊劃分，例如「區塊 1｜定義費氏數函式（迭代法）」等，簡潔地說明每個區塊的目的。

Gemini的Markdown 描述更為詳細和解釋性，不僅說明了函式的功能，還包含了演算法的分析（時間複雜度和空間複雜度），以及程式碼的具體實現細節（例如變數 a 和 b 的含義）。對於 main 函式的描述，也更詳細地解釋了每個步驟的目的和實現方式（例如 try...except 的作用、isdigit() 的用途）。對於程式進入點 if __name__ == "__main__":，提供了更深入的解釋，說明了其在 Python 模組化中的作用。

總體來說，Gemini的程式碼和 Markdown 傾向於提供更全面的教學和解釋，適合用於學習和理解程式碼，但是產出結果的細節方面有

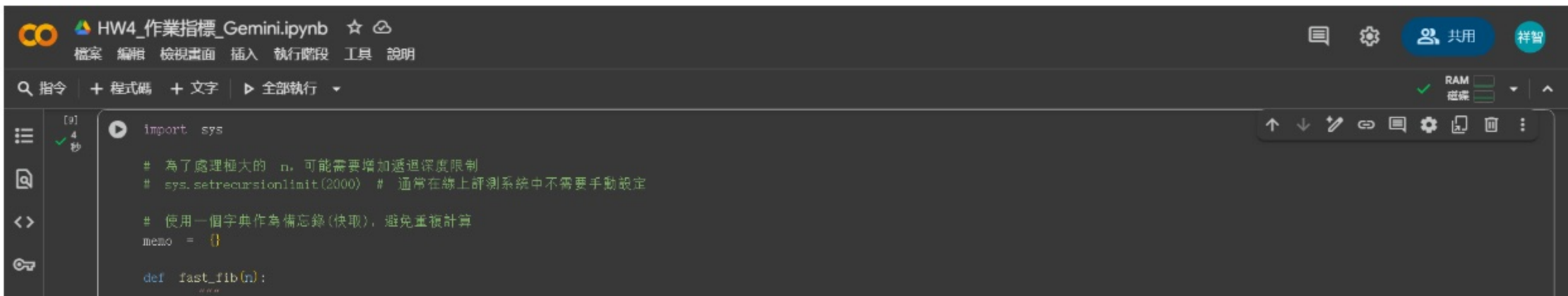
時會出錯，無法滿足prompt的規定以及它自己本身生成的結果說明；而GPT的描述更像是一個簡潔的程式結構概覽，為prompt提供一個最符合條件與要求的內容範本。

四、延伸比較

由於一開始的prompt將演算法固定為迭代法，故就算兩邊模型的編碼邏輯架構略有差異，但因為演算法條件固定所以運作效率感受不出差異，因此接下來追加prompt為"無限制任何演算法，也不限制只能輸入90以下的自然數，以上述追加條件重新生成能最快計算輸入值答案的程式"

五、生成結果(2)

Gemini:



```
        使用快速費氏數倍增法 (Fast Doubling) 計算第 n 個費氏數。
        這個輔助函式會回傳一個元組 (F(n), F(n+1))。
        時間複雜度: O(log n)
        """
        # 基本情況
        if n == 0:
            return (0, 1)

        # 檢查備忘錄中是否已存在結果
        if n in memo:
            return memo[n]

        # 遞迴步驟，先計算 n/2 的結果
        # a = F(k), b = F(k+1) where k = n/2
        k = n // 2
        a, b = fast_fib(k)

        # 應用倍增公式
        # c = F(2k) = a * (2*b - a)
        # d = F(2k+1) = a^2 + b^2
        c = a * (2 * b - a)
        d = a * a + b * b

        # 根據 n 的奇偶性回傳結果
        if n % 2 == 0:
            # 如果 n 是偶數 (n=2k)，我們需要 (F(2k), F(2k+1))，也就是 (c, d)
            result = (c, d)
        else:
            # 如果 n 是奇數 (n=2k+1)，我們需要 (F(2k+1), F(2k+2))
            # F(2k+1) 是 d，F(2k+2) = F(2k) + F(2k+1) = c + d
            result = (d, c + d)

        # 將結果存入備忘錄後回傳
        memo[n] = result
        return result

    # --- 主程式邏輯 ---

    # 讀取標準輸入的字串
    input_str = input()

    # 步驟 1: 判斷輸入是否為有效的非負整數
    if not input_str.isdigit():
        # 如果不是一個有效的非負整數，直接輸出 0
        print(0)
    else:
        # 步驟 2: 將有效的字串轉換為整數
        n = int(input_str)

        # 步驟 3: 呼叫高效的費氏數列函式
        # fast_fib(n) 回傳的是 (F(n), F(n+1))，我們只需要第一個元素 F(n)
        fib_n, _ = fast_fib(n)
        print(fib_n)
```

GPT-5:

(失敗):

HW4_作業指標_GPT-5.ipynb

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

共用 祥智

指令 + 程式碼 + 文字 全部執行

RAM 磁碟

[9] 0 秒

```python
# ## 區塊 1｜高效費氏數（快倍增，迭代版，O(log n)）
def fib(n: int) -> int:
    a, b = 0, 1    # a = F(k), b = F(k+1)
    for i in range(n.bit_length() - 1, -1, -1):
        # 雙倍步驟：從 (F(k), F(k+1)) 推到 (F(2k), F(2k+1))
        two_b_minus_a = (b << 1) - a                # 2*F(k+1) - F(k)
        c = a * two_b_minus_a                       # F(2k)
        d = a * a + b * b                           # F(2k+1)

        if (n >> i) & 1 == 0:
            a, b = c, d                             # 目標位元為 0 → (2k)
        else:
            a, b = d, c + d                         # 目標位元為 1 → (2k+1)
    return a


# ## 區塊 2｜輸入驗證（僅接受 ASCII 數字）
def is_valid_nonneg_int_ascii(s: str) -> bool:
    return bool(s) and all('0' <= ch <= '9' for ch in s)


# ## 區塊 3｜主程式：讀取 → 驗證 → 計算/輸出
def main():
    import sys
    s = sys.stdin.readline().strip()

    if is_valid_nonneg_int_ascii(s):
        n = int(s)
        print(fib(n))
    else:
        print(0)


# ## 區塊 4｜程式進入點
if __name__ == "__main__":
    main()
```

0

變數 終端機 晚上10:02 Python 3

(成功):

HW4_作業指標_GPT-5.ipynb

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

共用 祥智

指令 + 程式碼 + 文字 全部執行

RAM 磁碟

[28] 2秒

```python
import sys

# 在 Python 3.11+ 中解除超長整數字串解析限制（Colab 可能是 3.10 或 3.11，無此 API 則忽略）
try:
    sys.set_int_max_str_digits(0)
except Exception:
    pass

def fib_fast_doubling(n: int) -> int:
    # 迭代式 fast doubling，時間複雜度 O(log n)
    if n == 0:
        return 0
    a, b = 0, 1   # a = F(k), b = F(k+1)
    mask = 1 << (n.bit_length() - 1)
    while mask:
        d = a * ((b << 1) - a)    # F(2k)
        e = a * a + b * b         # F(2k+1)
        if n & mask:
            a, b = e, d + e       # (F(2k+1), F(2k+2))
        else:
            a, b = d, e           # (F(2k), F(2k+1))
        mask >>= 1
    return a

def main():
    try:
        s = input().strip()
    except EOFError:
        print(0)
        return

    # 僅接受非負整數（自然數含 0）
    if s and s.isdigit():
        n = int(s)    # 任意位數
        print(fib_fast_doubling(n))
    else:
        print(0)

if __name__ == "__main__":
    main()
```

變數 終端機

晚上11:36 Python 3

HW4_作業指標_GPT-5.ipynb

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

共用 祥智

指令 + 程式碼 + 文字 全部執行

RAM 磁碟

[28] 2秒

```python
try:
    sys.set_int_max_str_digits(0)
except Exception:
    pass

def fib_fast_doubling(n: int) -> int:
    # 迭代式 fast doubling，時間複雜度 O(log n)
    if n == 0:
        return 0
    a, b = 0, 1   # a = F(k), b = F(k+1)
    mask = 1 << (n.bit_length() - 1)
    while mask:
        d = a * ((b << 1) - a)    # F(2k)
        e = a * a + b * b         # F(2k+1)
        if n & mask:
            a, b = e, d + e       # (F(2k+1), F(2k+2))
        else:
            a, b = d, e           # (F(2k), F(2k+1))
        mask >>= 1
    return a
```

```
def main():
    try:
        s = input().strip()
    except EOFError:
        print(0)
        return

    # 僅接受非負整數（自然數含 0）
    if s and s.isdigit():
        n = int(s)   # 任意位數
        print(fib_fast_doubling(n))
    else:
        print(0)

if __name__ == "__main__":
    main()
```

```
1000
43466557686937456435688527675040625802564660517371780402481729089536555417949051890403879840079255169295922593080322634775209689623239873322471161642996440906533187938298969649928516003704476137
```

變數 終端機 晚上11:36 Python 3

LMArena

https://lmarena.ai/c/7f38f00d-1617-4ce4-9181-e4ae501510e1

LMArena

New Chat

Leaderboard

Today

費氏數列是一個數列...

Problem Descriptio...

hanly13731@gmail.com

Terms of Use Privacy Policy Cookies

Side by Side gemini-2.5-pro vs gpt-5-high

gemini-2.5-pro

好的，為了達到「最快」的計算速度，我們必須選用時間複雜度為 **O(log n)** 的演算法。對於計算費氏數列，**快速費氏數倍增法 (Fast Doubling)** 通常被認為是實作上最快的方法之一，因為它相比矩陣法有更小的常數開銷。

這個演算法的原理是利用以下兩個恆等式，從 `F(k)` 和 `F(k+1)` 直接跳到 `F(2k)` 和 `F(2k+1)`，從而讓計算步驟從 `n` 次大幅減少到約 $\log_2(n)$ 次。

1. $F(2k) = F(k) * [2*F(k+1) - F(k)]$
2. $F(2k+1) = F(k+1)^2 + F(k)^2$

以下是基於此演算法的 Python 程式。

gpt-5-high

```
        # Fast Doubling 公式
        d = a * ((b << 1) - a)  # F(2k)
        e = a * a + b * b       # F(2k+1)
        if n & mask:
            a, b = e, d + e     # 進到 2k+1
        else:
            a, b = d, e         # 進到 2k
        mask >>= 1
    return a

def main():
    data = sys.stdin.buffer.read()
    if not data:
        sys.stdout.write("0\n")
        return
```

← Left is Better | It's a tie | Both are bad | Right is Better →

Ask followup...

Inputs are processed by third-party AI and responses may be inaccurate.

LMArena

LMArena

New Chat

Leaderboard

Today

費氏數列是一個數列...

Problem Descriptio...

hanly13731@gmail.com

Terms of Use Privacy Policy Cookies

Side by Side gemini-2.5-pro vs gpt-5-high

gemini-2.5-pro

程式說明

1. **演算法選擇**：
   - 我們選擇了 **O(log n)** 的「快速費氏數倍增法」，這是計算極大項次費氏數的頂級演算法。對於 `n=1,000,000`，它只需要大約 20 次遞迴步驟，而傳統迭代法則需要一百萬次迴圈。
2. **核心函式 `fast_fib(n)`**：
   - **回傳值**：為了讓遞迴能夠運作，函式不僅需要計算 `F(n)`，還需要計算 `F(n+1)`。因此，它回傳一個包含這兩項的元組 `(F(n), F(n+1))`。
   - **備忘錄 (Memoization)**：我們使用一個字典 `memo` 來快取計算結果。在函式開始時，會先檢查 `n` 的結果是否已經存在於

gpt-5-high

```
    a, b = 0, 1  # a = F(k), b = F(k+1)
    mask = 1 << (n.bit_length() - 1)
    while mask:
        # Fast Doubling 公式
        d = a * ((b << 1) - a)  # F(2k)
        e = a * a + b * b       # F(2k+1)
        if n & mask:
            a, b = e, d + e     # 進到 2k+1
        else:
            a, b = d, e         # 進到 2k
        mask >>= 1
    return a

def main():
    data = sys.stdin.buf read()
```

← Left is Better | It's a tie | Both are bad | Right is Better →

Ask followup...

Inputs are processed by third-party AI and responses may be inaccurate.

LMArena

https://lmarena.ai/c/7f38f00d-1617-4ce4-9181-e4ae501510e1

LMArena

New Chat

Leaderboard

Today

費氏數列是一個數列...

Problem Descriptio...

Side by Side gemini-2.5-pro vs gpt-5-high

gemini-2.5-pro

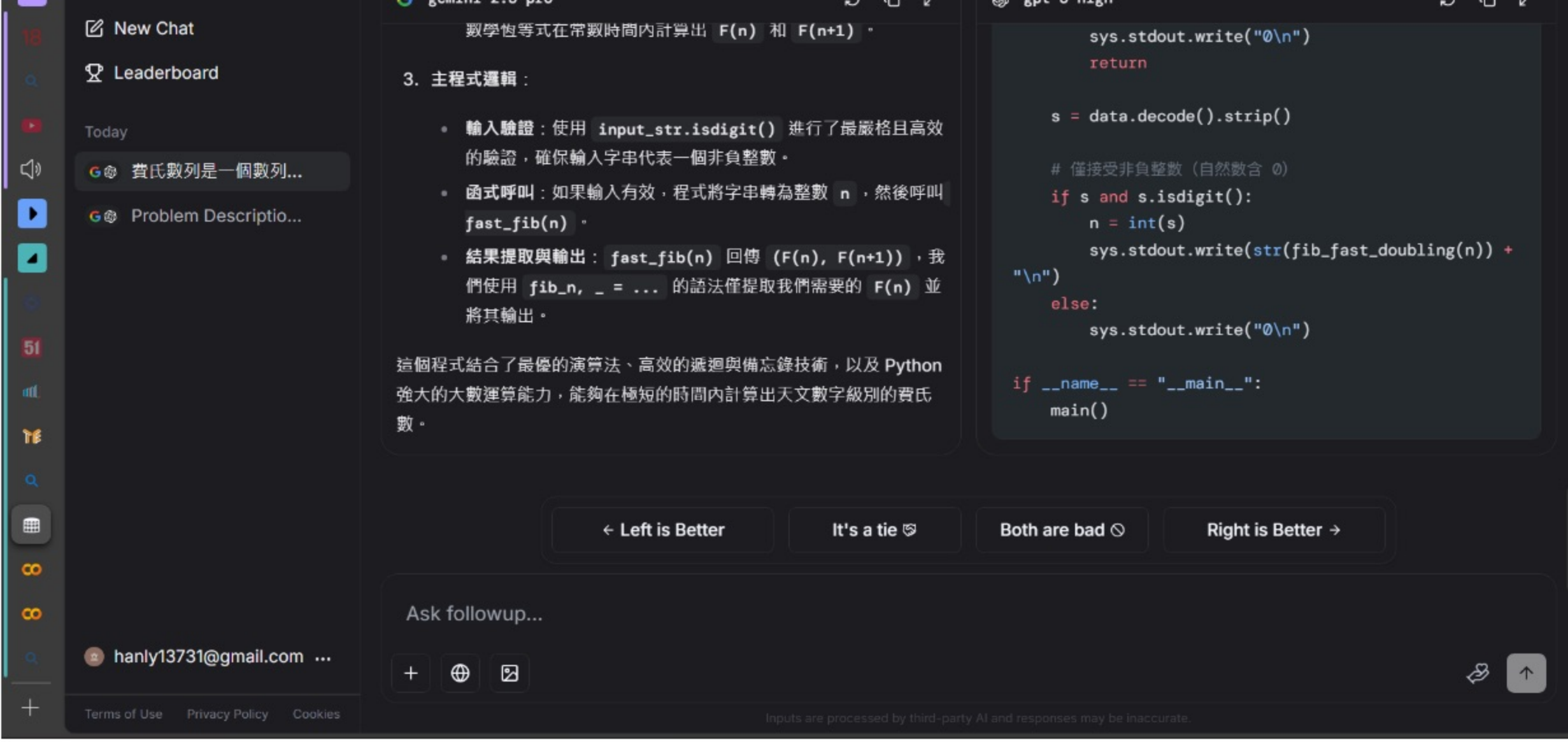
gpt-5-high

   - **備忘錄 (Memoization)**：我們使用一個字典 `memo` 來快取計算結果。在函式開始時，會先檢查 `n` 的結果是否已經存在於 `memo` 中。如果存在，就直接回傳，避免了指數級的重複計算。這是讓遞迴保持 O(log n) 效率的關鍵。
   - **遞迴**：函式的核心是遞迴呼叫 `fast_fib(n // 2)`，將問題規模減半。
   - **倍增計算**：拿到 `F(n//2)` 和 `F(n//2 + 1)` 的結果後，利用數學恆等式在常數時間內計算出 `F(n)` 和 `F(n+1)`。
3. **主程式邏輯**：
   - **輸入驗證**：使用 `input_str.isdigit()` 進行了最嚴格且高效的驗證，確保輸入字串代表一個非負整數。

```
    data = sys.stdin.buffer.read()
    if not data:
        sys.stdout.write("0\n")
        return

    s = data.decode().strip()

    # 僅接受非負整數（自然數含 0）
    if s and s.isdigit():
        n = int(s)
        sys.stdout.write(str(fib_fast_doubling(n)) +
"\n")
    else:
```



從追加生成的結果來看，兩個模型大抵上皆維持原本的風格--Gemini註解詳盡且程式塊拆分較繁瑣但理解難度也較低，甚至在程式以外的生成內容中還十分仔細地說明技術原理、演算法選擇和程式邏輯等說明，的確是適合用來學習程式語言和數學模型的工具；GPT則是一個口令一個動作，在提示所能生成的可能結果中，選擇最為簡潔、最符合要求且運行錯誤的防範機制較佳的生成結果；但不知為何我再次提出延伸的prompt時，GPT始終無法生成能在

式邏輯等說明，的確是適合用來學習程式語言和數學模型的工具；GPT則是一個口令一個動作，在提示所能生成的可能結果中，選擇最為簡潔、最符合要求且運行錯誤的防範機制較佳的生成結果；但不知為何我再次提出延伸的prompt時，GPT始終無法生成能在python3正常運行的程式塊，待我將prompt微調為"能在colab上順利運行的python程式碼"後才成功。

兩者在計算器最優解的演算法選擇上不約而同選擇了"時間複雜度O(log n)"以取代原先較方便簡單的迴圈迭代法，顯示兩者對於數學模型的理解以及資料蒐集的準確度上實力相當，而Gemini的生成速度及內容質量上依舊勝於GPT-5。

整體而言，在程式生成的能力方面，若是把LLM單純當成程式生成器的話，GPT-5是較為高效直截的選擇，但若是今天要學習一門程式語言或數學模型，乃至於程式運行錯誤的除錯階段，Gemini是更為有用的支援工具。